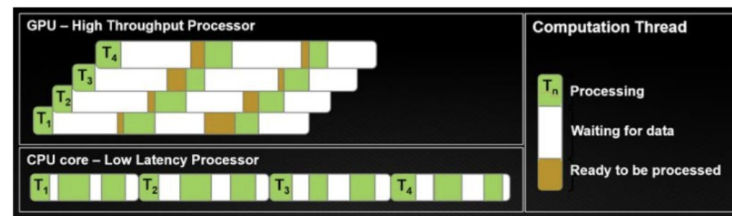
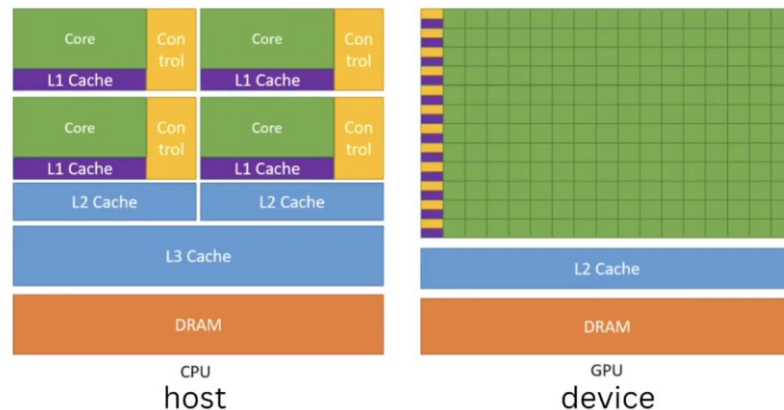


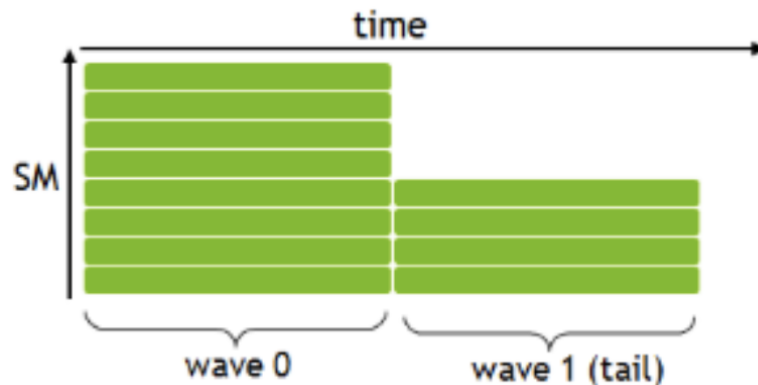
What a GPU is good at

- CPUs are good for
 - Sequential operations
 - Unrelated tasks
 - Flexibility
 - High clock frequency
 - Few fast threads
 - Latency: each thread finishes quickly
- GPUs are good for
 - Running a large number of small similar operations (threads) in parallel
 - More transistors for data processing (ALUs)
 - Lower clock frequency BUT can run the same operation on many data inputs at the same time
 - Throughput: total processed data
- Huge execution speedup for certain tasks



Hardware and execution interact

- Threads are grouped into consecutive blocks and scheduled onto SM in waves
- We want to ensure waves have an equal amount of computation
- Problem: last wave has fewer thread blocks, leaving some SMs idle (low occupancy)
- **Wave quantization:** make number of thread blocks divide # SMs.
- Rule of thumb: number of thread blocks should be $\geq 4 \times \# \text{ SMs}$
- Challenge: some aspects of hardware are hidden from the execution model (e.g., scheduling, # SMs)



Anatomy of a GPU - SMs

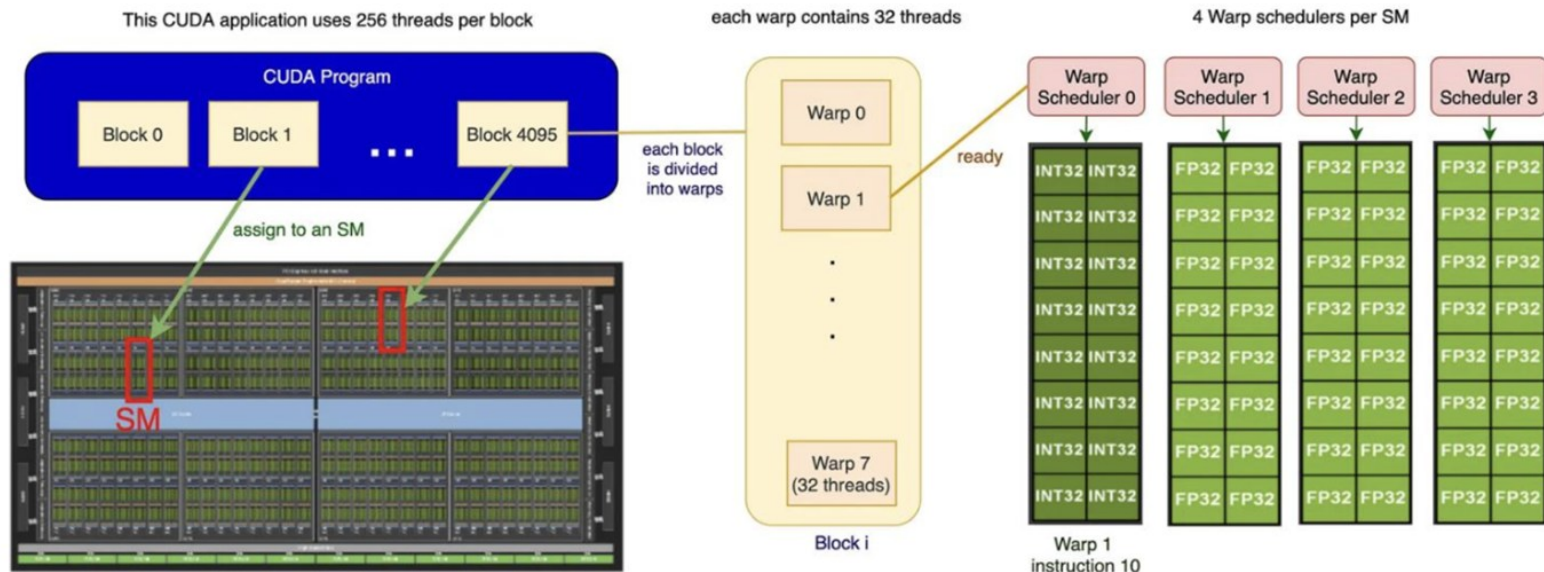


SM



GA100 Full GPU with 128 SMs

GPU execution model

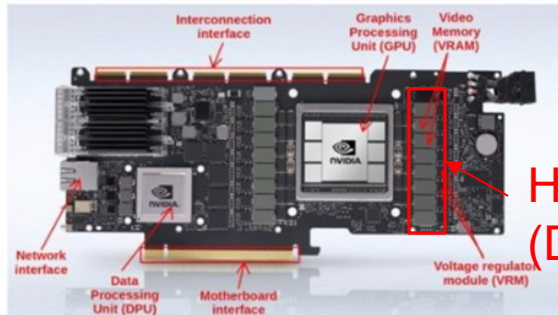


Connection to HBM

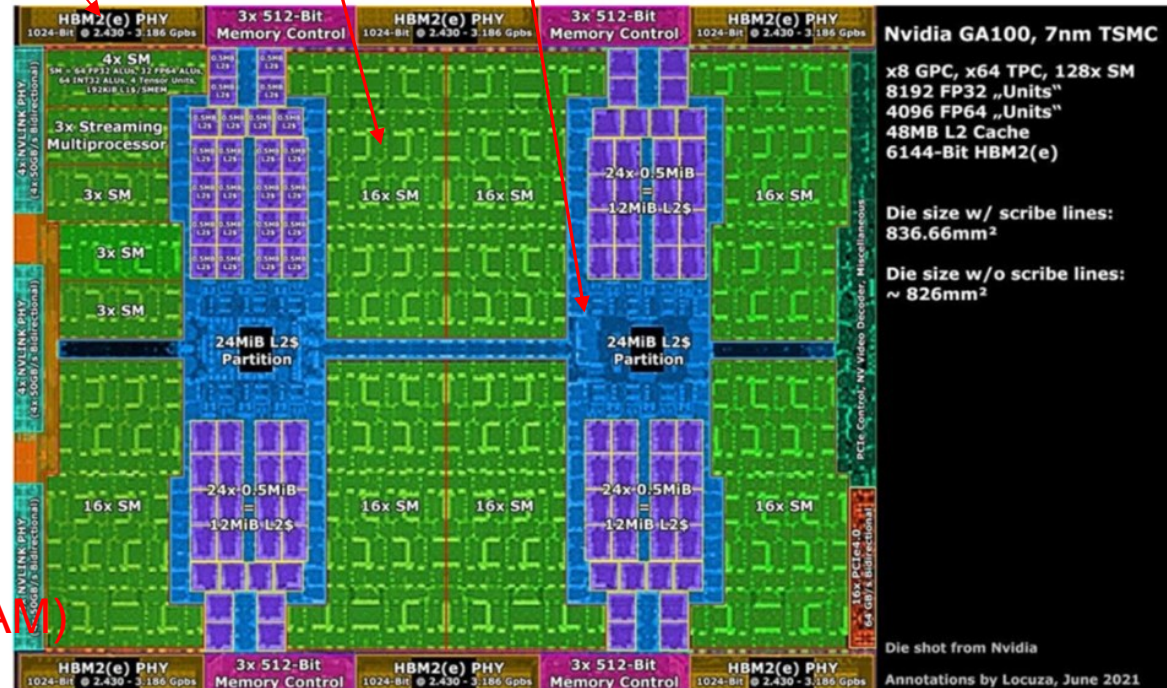
L1 (SRAM) L2 (SRAM)

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	CPI (cycles)
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)



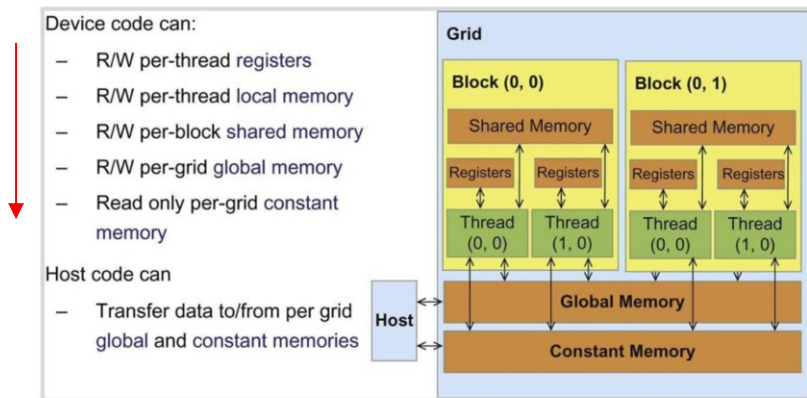
HBM
(DRAM)



Memory model of a GPU

- Each thread can access its own register and shared memory within the block (fast)
- Information going through the blocks needs to be written/read from global memory (slow)

slower



TPU high level architecture

GPUs, TPUs, and many other accelerators are at a high level, similar

